

Actividad: Implementación de un Traductor Automático con RNN bajo el estándar CRISP-ML(Q)

1. Identificación de la Actividad

- **Tema:** Arquitecturas de Redes Neuronales Recurrentes (RNN/LSTM/GRU) para Procesamiento de Lenguaje Natural (NLP).
- **Metodología:** CRISP-ML(Q) (Cross-Industry Standard Process for Machine Learning with Quality Assurance).
- **Desafío:** Desarrollar un prototipo de traductor **Español ↔ Quechua** (o Inglés) para la comunidad universitaria de la **UAC**.

2. Guía de Implementación por Fases

Fase 1: Business and Data Understanding

Los equipos deben definir el alcance técnico y las métricas de éxito antes de tocar una sola línea de código.

- **Tarea:** Definir la métrica de desempeño principal (ej. **BLEU Score**) y la latencia máxima permitida para la traducción de una frase de 20 palabras.
- **Análisis de Datos:** Evaluar el corpus disponible. ¿Existen suficientes parejas de frases? ¿Cuál es la longitud máxima de las secuencias?

Fase 2: Data Preparation (Crucial para RNN)

El preprocesamiento es donde la mayoría de los modelos de RNN fallan si no hay calidad.

- **Tokenización y Padding:** Implementar la limpieza de texto (minúsculas, eliminar puntuación) y la creación de vocabularios.
- **Vectorización:** Uso de Word Embeddings (propios o pre-entrenados como FastText).
- **QA de Datos:** Verificar que no existan sesgos lingüísticos o traducciones vacías que "ensucien" la memoria de la red.

Fase 3: Modeling (Arquitectura Seq2Seq)

- **Construcción:** Diseñar una arquitectura **Encoder-Decoder**.
- **Elección de Celda:** Justificar el uso de **LSTM** o **GRU** para mitigar el problema del gradiente desvaneciente.

- **Mecanismo de Atención (Opcional/Avanzado):** Implementar atención para mejorar la traducción en frases largas.

Fase 4: Evaluation

- **Validación:** Comparar el modelo contra un conjunto de prueba (*Hold-out set*).
- **Análisis de Errores:** Identificar si el modelo falla por falta de vocabulario (OOV - *Out Of Vocabulary*) o por pérdida de contexto en secuencias largas.

Fase 5: Deployment

- **Exportación:** Convertir el modelo a un formato ligero (ej. TensorFlow Lite o ONNX).
- **Interfaz:** Crear un microservicio (Flask/FastAPI) que reciba una cadena de texto y devuelva la traducción.

Fase 6: Monitoring and Maintenance (La "Q" de CRISP-ML)

- **Detección de Deriva:** ¿Cómo sabremos si el traductor empieza a fallar cuando los alumnos usan jerga universitaria actual?
- **Feedback Loop:** Diseñar un botón de "Sugerir corrección" para recolectar datos y re-entrenar el modelo en el futuro.

3. Entregables del Proyecto

1. **Documento Maestro CRISP-ML(Q):** Un PDF detallando cada fase, con especial énfasis en el control de calidad aplicado.
2. **Notebook de Entrenamiento:** Código documentado con el preprocesamiento, la arquitectura de la RNN y las curvas de pérdida (*Loss*) y precisión.
3. **Prototipo Funcional:** Enlace a un repositorio o demo donde se pueda ingresar una frase y obtener la traducción.

4. Rúbrica de Evaluación (0-20)

Criterio	Logrado (17-20)	En Proceso (11-16)	No Logrado (0-10)
Arquitectura RNN	Implementa un Encoder-Decoder funcional con manejo correcto de estados ocultos.	La red entrena pero tiene problemas graves de <i>overfitting</i> o <i>padding</i> incorrecto.	No logra estructurar la red para secuencias.
Metodología CRISP-ML	Documenta rigurosamente las 6 fases, incluyendo planes de monitoreo.	Solo se enfoca en el entrenamiento (Fase 3) y descuida el negocio y mantenimiento.	Ignora las fases de la metodología.
Calidad de Traducción	El BLEU score es aceptable y el modelo maneja bien frases cortas/medianas.	La traducción es literal o pierde el sentido frecuentemente por mal preprocesamiento.	El modelo produce texto incoherente.
Despliegue y QA	Presenta una API funcional con validación de entradas.	El modelo solo corre localmente en el notebook de entrenamiento.	No hay producto entregable fuera del código.